

PD-TDM: TEMPORAL MULTIPLEXING OF PREFILL AND DECODE FOR SLO-AWARE LLM SERVING

[Author list pending]

[Affiliations pending — internal draft]

ABSTRACT

Chunked prefill bounds the prefill work admitted into an iteration, but the service share between prefill and decode remains implicit in mixed batches. This is restrictive when one tensor-parallel replica must serve both phases and neither another replica nor intra-device partitioning is available. We present PD-TDM, which retains bounded prefill and local KV state while scheduling prefill-only and decode-only iterations with an explicit phase share. On Qwen3-8B using two Ascend 910B3 NPUs, PD-TDM improves strict joint-SLO attainment over same-platform vLLM-Ascend chunked prefill by 34.8 percentage points on a conversation burst and 73.9 points on a code burst at representative high loads. A reconstructed MaaS trace shows lower mean/P99 time to first token. Decode-heavy tests expose the boundary: benefits diminish or reverse when decode has little latency slack.

Index Terms— Large language model serving, prefill–decode scheduling, temporal multiplexing, service-level objectives, Ascend NPU

1. INTRODUCTION

Online large-language-model (LLM) serving must control both time to first token (TTFT) and time per output token (TPOT). Prefill processes the prompt and is typically compute intensive; decode repeatedly reads a growing KV cache and is often memory intensive. A scheduler therefore needs to advance queued prompts without creating long gaps for active generations.

Continuous batching rebuilds the batch at every iteration [1, 2]. Chunked prefill further limits prompt work and mixes a bounded prefill chunk with decode tokens, reducing generation stalls [3, 4]. This bound controls how much prefill work enters one iteration, but does not independently specify how often prefill is served: under a decode-first token budget, available prefill capacity depends on the current decode batch.

Physical prefill–decode disaggregation can provision the phases independently, but requires distinct instances and KV transfer [5, 6]. Spatial multiplexing instead depends on concurrent kernels and controllable compute partitioning. We study the complementary deployment budget in which one tensor-parallel (TP) replica occupies the allocated devices and both phases retain its weights and KV cache locally.

We present PD-TDM, a scheduler that separates state sharing from execution. It preserves chunked continuation, but each model iteration is prefill-only or decode-only. A selector exposes the long-run prefill service tendency as a configurable share, and work-conserving fallback serves the nonempty phase. This creates an explicit interface between TTFT and TPOT pressure. The benefit is conditional: prefill-only slots can reduce queueing when decode has TPOT slack, whereas they can harm a saturated decode queue.

Our contributions are: (1) identifying the chunk bound and cross-iteration phase share as distinct controls; (2) implementing bounded, phase-pure scheduling in a shared TP replica without weight duplication or KV transfer; and (3) evaluating both its gains and boundary on two Ascend 910B3 NPUs. Three-seed Azure-derived bursts show strict joint-SLO improvements, a large reconstructed MaaS workload supports the TTFT trend, and decode-heavy cases show that PD-TDM is not a universal replacement for mixed chunked prefill.

2. BACKGROUND AND RELATED WORK

Aggregated serving. Orca introduced iteration-level scheduling, and vLLM’s PagedAttention enables memory-efficient continuous batching [1, 2]. Long prefills can nevertheless delay generation. Sarathi-Serve bounds prefill chunks and constructs decode-maximal mixed batches; Dynamic SplitFuse similarly splits and combines prompt and generation work [3, 4]. PD-TDM retains this prior bounded-prefill contribution. Its additional control is the opportunity assigned to each phase across iterations. Our measured baseline is the native chunked-prefill path in vLLM-Ascend on the same NPUs, denoted vLLM-Ascend CP; we do not claim a cross-platform comparison to the Sarathi-Serve system.

Disaggregation and spatial multiplexing. Splitwise and DistServe place prefill and decode on different instances, while Mooncake builds a KV-cache-centric distributed architecture [5–7]. These designs gain independent phase capacity at the cost of additional replicas and state movement. MuxWise and semi-PD share storage while concurrently partitioning GPU compute between the phases [8, 9]. Their spatial isolation is orthogonal to PD-TDM, which gives the entire TP replica to one phase at a time.

Temporal orchestration. EcoServe is the closest prior de-

sign: it temporally disaggregates phases within an instance and rolls activation across multiple cooperating instances to keep prefill available [10]. PD-TDM does not claim to originate temporal disaggregation. It instead studies bounded, iteration-granular phase scheduling when the deployment has only one shared TP replica, and explicitly reports the decode-pressure region in which this choice fails.

3. PROBLEM AND METRIC

Let an iteration have token budget B , prefill bound C , and D_t active decode tokens. A decode-first mixed scheduler can admit prefill work $P_t \leq \min(C, B - D_t)$. Thus C bounds a chunk, but the current decode batch implicitly determines the prefill service left in that iteration. PD-TDM separates the controls: a prefill iteration admits at most $P_t \leq \min(C, B)$ prompt tokens, while a phase share determines how frequently prefill is selected when both queues contain work.

We evaluate this exchange with request-level joint-SLO attainment and goodput. For requests arriving in measurement window T , output length o_i , and thresholds S_T, S_P , we use

$$G = \frac{1}{T} \sum_i o_i \mathbf{1}[\text{TTFT}_i < S_T \wedge \text{TPOT}_i < S_P]. \quad (1)$$

Here TPOT is the per-request mean inter-token time, not a bound on every token interval. Goodput can differ at similar raw throughput when latency distributions fall on different sides of the joint threshold.

4. PD-TDM DESIGN

PD-TDM runs inside one model instance on the same TP devices. Weights and KV blocks remain local. Before every model iteration, the scheduler selects phase P or D . A P iteration advances only unfinished prompts; a D iteration generates one token for each selected active sequence. This phase-purity invariant is enforced for running partial prefills and newly admitted requests.

Bounded continuation. The prefill planner applies an aggregate $C = 2,048$ token limit per P iteration. It first continues partial prefills and then admits waiting requests with the remaining budget. An unfinished prompt retains its KV state and continues later. The token bound limits admitted work; since iteration time is workload dependent, it is not a strict wall-clock guarantee.

Phase selection. A token bucket accumulates a configurable target prefill share $r \in [0, 1]$. When both phases have work, sufficient credit selects P and consumes one unit; otherwise it selects D . If only one phase has work, the scheduler immediately serves it, so r describes a contention-time tendency rather than a fixed sequence or wall-clock ratio. The implementation also exposes TTFT-urgency and decode-starvation overrides. Our claims concern the observed configured scheduler, not an online controller that finds an optimal r .

Table 1. Supporting MaaS replay results; latency in ms.

Metric	vLLM-Ascend CP	PD-TDM
TTFT mean / P99	952 / 1,316	808 / 1,180
TPOT mean / P99	236 / 257	222 / 252
3× SLO attainment	99.997%	99.997%
1.2× attainment*	63.4%	92.6%
1.2× goodput*	99.9	150.5

*Post-hoc; goodput in output token/s.

This design predicts its boundary. When prompt queueing dominates and TPOT has slack, concentrated P iterations can move TTFT left while D iterations remain within the TPOT threshold. Under a deep decode queue, the silence created by a P iteration consumes the remaining TPOT slack and mixed chunked prefill can be preferable.

5. EVALUATION

Setup. We use Qwen3-8B in BF16 on two Ascend 910B3 NPU (64 GB each) with TP=2, CANN 8.3.rc1, PyTorch/torch-npu 2.7.1, and vLLM-Ascend v0.11.0rc1. The direct baseline, vLLM-Ascend CP, and PD-TDM use the same replica and 2,048-token prefill bound. T6 replays Azure-derived conversation and code bursts with a 10-s cycle. Mean prompt/output lengths are 1,155/211 and 2,048/28 tokens, respectively. Each point runs for 90 s with a 30 s warm-up; we report the median of three arrival seeds. Strict TTFT/TPOT thresholds are 200/120 ms for conversation and 500/200 ms for code. Metrics include requests whose arrivals fall in the measurement cohort. We use only runs collected after fixing the scheduler’s waiting gate.

Azure bursts. Fig. 1 shows the load sweep. At conversation scale $k = 3.0$, PD-TDM raises attainment from 29.0% to 63.8% (+34.8 points) and goodput from 1,525 to 3,396 output token/s. At code scale $k = 4.9$, attainment rises from 6.5% to 80.4% (+73.9 points) and goodput from 87 to 438 output token/s. The large relative goodput at this second point should be read with its low baseline attainment, not as a workload-independent speedup. Wider SLOs often place both systems near 100% attainment, indicating that the main effect is a latency-distribution shift rather than unconditional raw-throughput growth.

Scheduler telemetry supports the execution claim without establishing a kernel cause. In one persistently backlogged code point, PD-TDM alternates pure P/D iterations and the decode-iteration inter-arrival P50/P99 is 189/203 ms. Mixed and pure iterations perform unequal work, so their durations do not establish a fixed mixed-kernel tax.

MaaS replay and boundary. We reconstruct request lengths from minute-level aggregates and replay a 76-minute arrival trace containing 35,441 submitted requests per system, with no client errors. The configured cohort window is

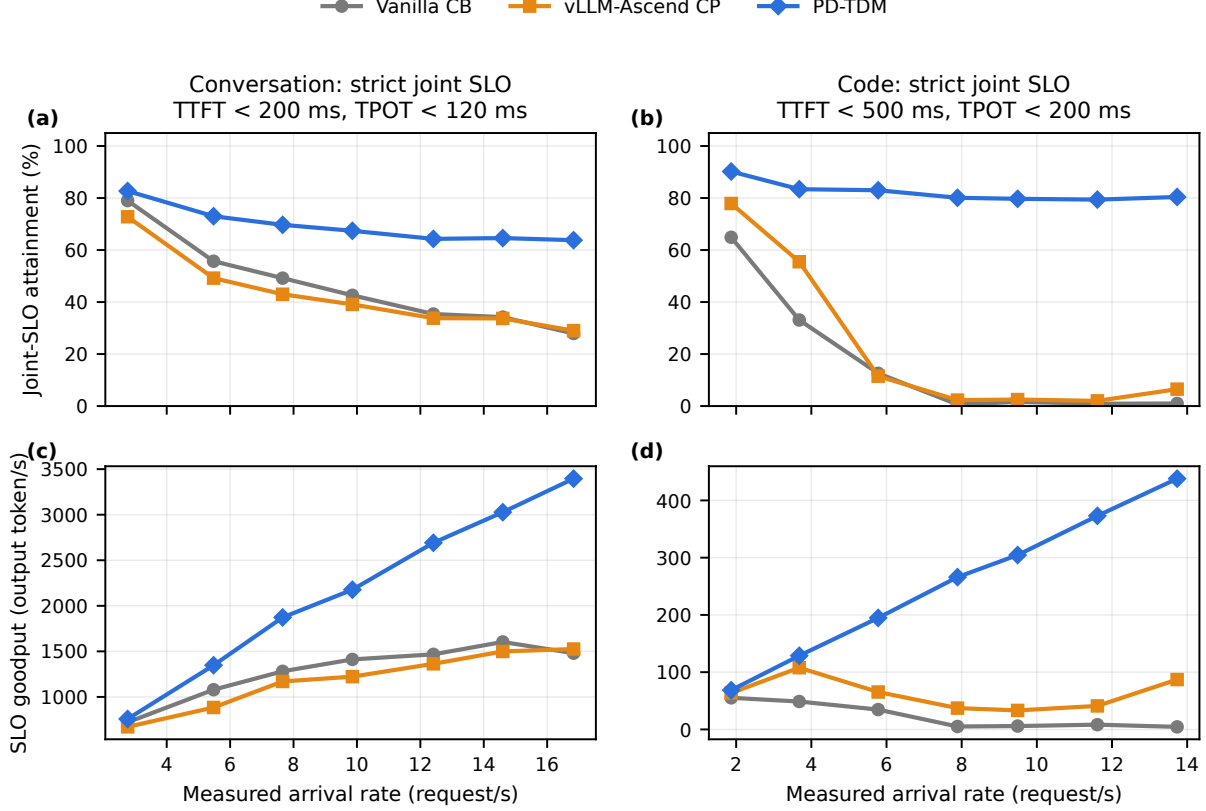


Fig. 1. Strict joint-SLO attainment and output-token goodput on Azure-derived bursts. Each point is the median of three seeds. vLLM-Ascend CP and PD-TDM use the same two-NPU TP replica and 2,048-token prefill bound.

[30, 34200) s; it contains 35,345 arrivals, of which 21 baseline and 3 PD-TDM records lack matched latency telemetry. Table 1 reports matched-record latency summaries. PD-TDM lowers mean/P99 TTFT by 15.1%/10.4%. Under the original loose $3\times$ ideal SLO, both systems are at the ceiling. Tightening the same traces to $1.2\times$ ideal produces the listed separation, but this is a post-hoc sensitivity analysis rather than a prespecified service target.

Synthetic workloads expose the other side of the trade-off. Balanced loads show TTFT gains, whereas longer-output, decode-heavy cells become neutral and then favor the mixed baseline at high load. These boundary runs are single seed and some high-load cells have client timeouts, so we treat them as directional counterexamples rather than stable capacity estimates.

Limitations. The evidence is limited to Qwen3-8B, one two-NPU platform, one software version, and a fixed 2,048-token bound. We have not completed best-tuned baseline chunk sensitivity, controlled same-backend mixed/pure isolation, static-ratio sensitivity, or cross-model validation. The MaaS input is reconstructed from aggregates and its tighter SLO is post hoc. The results therefore support an end-to-end comparison of these evaluated deployments, but do not isolate

a kernel-level cause or establish a universally optimal phase share.

6. CONCLUSION

PD-TDM separates a bounded prefill chunk from the cross-iteration service share between prefill and decode. It keeps weights and KV state in one shared TP replica while scheduling phase-pure iterations. On two Ascend 910B3 NPUs, this design improves joint-SLO attainment for prefill-heavy bursts and shifts TTFT left on a reconstructed MaaS trace. Decode-heavy counterexamples provide the necessary qualification: temporal phase isolation is useful when decode has latency slack, and can lose when TPOT is already tight.

7. REFERENCES

- [1] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 521–538.

- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee, “Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 117–134.
- [4] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He, “DeepSpeed-FastGen: High-throughput text generation for LLMs via MII and DeepSpeed-Inference,” *arXiv preprint arXiv:2401.08671*, 2024.
- [5] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 118–132.
- [6] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang, “Dist-serve: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 193–210.
- [7] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu, “Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot,” in *23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025, pp. 155–170.
- [8] Yukang Chen, Weihao Cui, Han Zhao, Ziyi Xu, Xiaozhe Fan, Xusheng Chen, Yangjie Zhou, Shixuan Sun, Bingsheng He, and Quan Chen, “Towards high-goodput LLM serving with prefill-decode multiplexing,” *arXiv preprint arXiv:2504.14489*, 2026.
- [9] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qili Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, Yun Liang, and Yu Wang, “semi-PD: Towards efficient LLM serving via phase-wise disaggregated computation and unified storage,” *arXiv preprint arXiv:2504.19867*, 2025.
- [10] Jiangsu Du, Hongbin Zhang, Taosheng Wei, Zhenyi Zheng, Jiazhi Jiang, Kaiyi Wu, Zhiguang Chen, and Yutong Lu, “Efficient LLM serving on commodity GPU clusters with Data-Reduced Cross-Instance orchestration,” in *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026, pp. 1787–1802.